

Bundesliga Analyzer

Inhalt

1. Einleitung	1
2. Projektantrag	1
2.1 Projektantrag	1
2.2 Flowchart	3
3. Projektplanung	4
4. Zeitplanung	4
5. Beschreibung zur Implementierung und Umsetzung.....	4
6. Problembeschreibungen, Schwierigkeiten und Überarbeitungen	5
8. Screenshots von Code-Snippets	6
7. Screenshots vom Debugging	8
8. Lösungsansätze	9
9. Quellenangaben	9

1. Einleitung

Name: Mariia Bashyrova

Gruppe: FI U 25-4

Im Rahmen dieses Projekts wurde die Desktop-Anwendung „Bundesliga Analyzer“ entwickelt. Ziel der Anwendung ist die Analyse und Auswertung von Bundesliga-Daten auf Basis einer öffentlichen Fußball-API.

Die Anwendung ermöglicht das automatische Laden von Spiel- und Tabellendaten, die Speicherung der Informationen in einer SQLite-Datenbank sowie die Berechnung von Tabellenständen und Platzprognosen. Zusätzlich können Tabellenstände verschiedener Spieltage miteinander verglichen und die Ergebnisse als CSV-Datei exportiert werden.

2. Projektantrag

2.1 Projektantrag

Thema des Projekts

Entwicklung einer Desktop-Anwendung zur Analyse, Verwaltung und Auswertung von Bundesliga-Daten. Die Software soll aktuelle Spielinformationen aus einer öffentlichen Fußball-API abrufen, lokal speichern und verschiedene statistische Auswertungen sowie Prognosen zur Tabellenentwicklung bereitstellen.

Zielsetzung

Ziel des Projekts ist die Entwicklung einer funktionalen Softwareanwendung mit praktischem Nutzen für Fußballinteressierte. Der Benutzer soll Bundesliga-Daten automatisch aktualisieren, Spieltage und Tabellenstände anzeigen sowie statistische Analysen durchführen können.

Darüber hinaus soll die Anwendung die Entwicklung der Tabellenplatzierungen zwischen verschiedenen Spieltagen vergleichen und eine Prognose der möglichen Endplatzierung einer Mannschaft auf Basis der aktuellen Punktesituation berechnen.

Geplante Funktionen

- Abruf von Bundesliga-Daten über eine öffentliche API
- Automatische Aktualisierung von Spielergebnissen
- Speicherung der Daten in einer SQLite-Datenbank
- Anzeige von Spieltagen und Spielergebnissen
- Anzeige der aktuellen Ligatabelle
- Automatische Berechnung von Punkten, Torverhältnis und Tabellenplatzierungen
- Vergleich von Tabellenständen zwischen zwei Spieltagen
- Anzeige von Positionsveränderungen der Mannschaften (Aufstieg, Abstieg oder unveränderte Position)
- Statistische Auswertung der Mannschaften und der Liga
- Berechnung von Platzprognosen auf Basis aktueller Punktestände
- Such- und Filterfunktionen für Spieltage und Mannschaften
- Export von Tabellen- oder Statistikdaten als CSV-Datei

Datenhaltung

Die Daten werden in einer SQLite-Datenbank gespeichert.

Die Daten werden über eine öffentliche Fußball-API geladen und lokal gespeichert, um schnelle Auswertungen und Berechnungen zu ermöglichen.

Geplante Vorgehensweise

Zunächst wird die Projektstruktur erstellt und die Datenbank eingerichtet. Anschließend werden die Datenmodelle sowie die Datenbankzugriffe implementiert.

Danach erfolgt die Anbindung an die externe API, um Bundesliga-Daten automatisch abzurufen und in der Datenbank zu speichern.

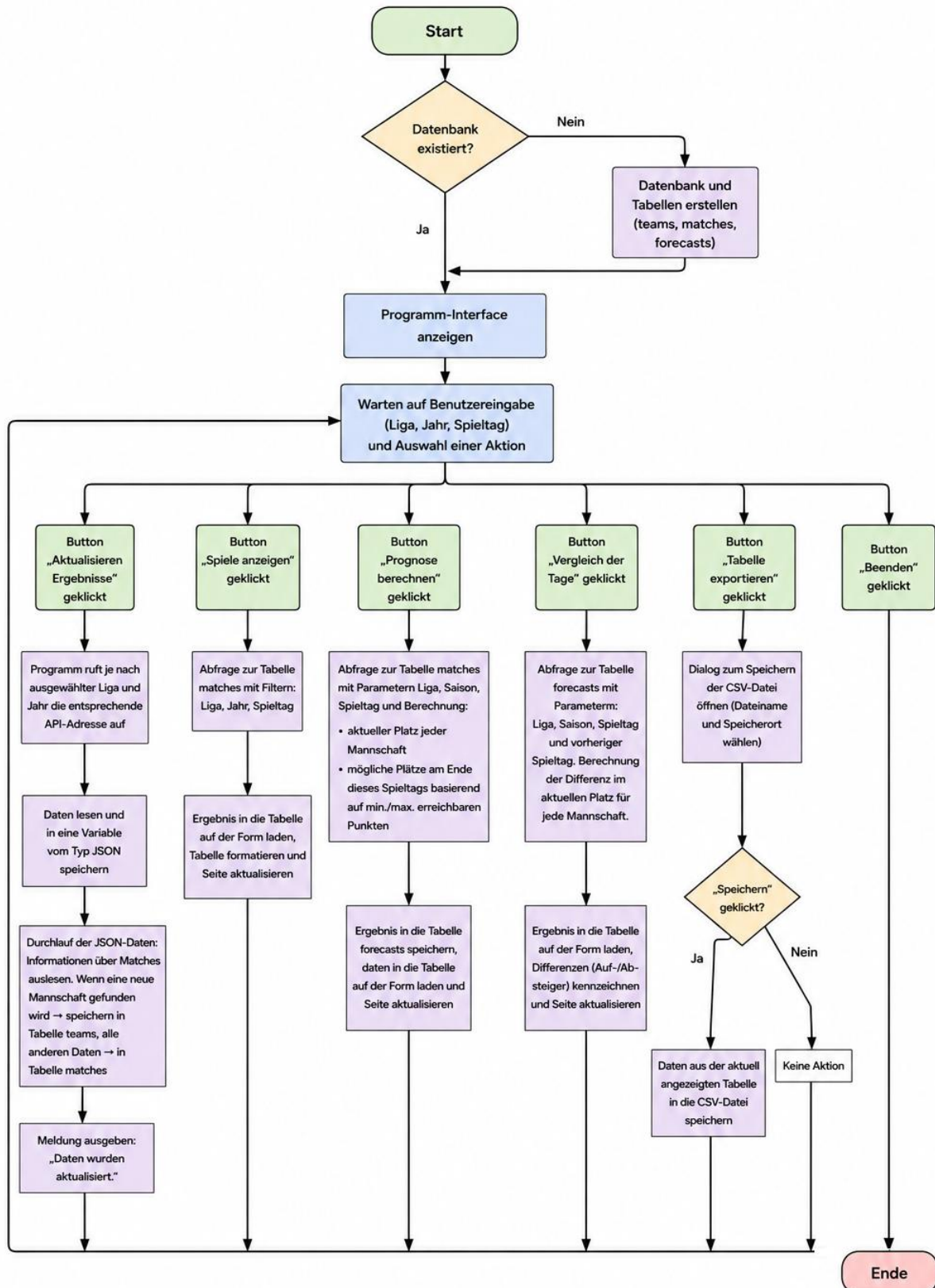
Im nächsten Schritt werden die Funktionen zur Anzeige von Spieltagen, Ergebnissen und Tabellenständen umgesetzt. Anschließend werden die Analysefunktionen zur Berechnung von Tabellenständen, Positionsveränderungen und Platzprognosen entwickelt.

Zum Abschluss werden die Exportfunktion, Tests der Anwendung sowie die Projektdokumentation und Präsentation erstellt.

Verwendete Technologien

- Programmiersprache: C#
- Benutzeroberfläche: Windows Forms (WinForms)
- Datenbank: SQLite
- Datenformat der API: JSON
- Entwicklungsumgebung: Visual Studio 2022

2.2 Flowchart



3. Projektplanung

Zu Beginn des Projekts wurde die grundlegende Struktur der Anwendung geplant. Anschließend wurde die Benutzeroberfläche mit Windows Forms erstellt.

Danach erfolgte die Entwicklung der SQLite-Datenbank. Im nächsten Schritt wurde die OpenLigaDB-API angebunden, um Bundesliga-Daten automatisch abzurufen.

Nach der erfolgreichen Datenübernahme wurden Funktionen zur Anzeige von Spieltagen und Tabellenständen entwickelt. Anschließend wurde die Berechnung von Tabellenprognosen implementiert und die Speicherung der Prognosen in der Datenbank ergänzt.

Danach wurde eine Vergleichsfunktion entwickelt, die die Veränderungen zwischen zwei Spieltagen analysiert. Zum Abschluss wurden eine CSV-Exportfunktion sowie verschiedene Test- und Debugging-Maßnahmen umgesetzt.

4. Zeitplanung

Tag	Aufgabe
1	Projektantrag und Planung
2	Erstellung der Benutzeroberfläche, Entwicklung der SQLite-Datenbank, Anbindung der OpenLigaDB-API, Verarbeitung und Speicherung der Daten
3	Anzeige von Spielen und Tabellen, Berechnung der Tabellenprognose
4	Speicherung der Prognosedaten, Vergleich von Spieltagen
5	Erstellung von Testdaten, CSV-Export
6	Tests, Fehlerbehebung, Screenshots machen, Dokumentation fertigstellen
7	Präsentation des Projekts

5. Beschreibung zur Implementierung und Umsetzung

Datum	Zeitraum	Beschreibung
15.06.2026	09:00–12:00	Die grafische Benutzeroberfläche wurde mit Windows Forms erstellt. Dabei wurden Auswahlfelder für Saison, Liga und Spieltag sowie mehrere Registerkarten für die verschiedenen Funktionen der Anwendung implementiert. Ziel war die Bereitstellung einer übersichtlichen und benutzerfreundlichen Oberfläche.
15.06.2026	13:00–14:00	Es wurde eine SQLite-Datenbank entwickelt. Hierfür wurden die Tabellen teams, matches und forecasts erstellt, um Mannschafts-, Spiel- und Prognosedaten dauerhaft speichern zu können. SQLite wurde gewählt, da keine zusätzliche Serverinstallation erforderlich ist.
15.06.2026	14:00–16:00	Die OpenLigaDB-API wurde angebunden. Mithilfe der Klasse HttpClient werden Bundesliga-Daten im JSON-Format geladen. Die Anbindung ermöglicht die automatische Aktualisierung der Daten ohne manuelle Eingaben.
15.06.2026	16:00–17:00	Die von der API geladenen JSON-Daten wurden verarbeitet und in die SQLite-Datenbank gespeichert. Hierfür wurden die benötigten Eigenschaften aus dem

Datum	Zeitraum	Beschreibung
		JSON-Dokument ausgelesen und in die entsprechenden Datenbanktabellen übertragen.
16.06.2026	09:00–12:00	SQL-Abfragen zur Anzeige von Spieltagen, Mannschaften und Tabellenständen wurden entwickelt. Die Ergebnisse werden in DataGridView-Elementen dargestellt, damit der Benutzer die Daten übersichtlich analysieren kann.
16.06.2026	13:00–15:00	Die Darstellung der Benutzeroberfläche wurde optimiert. Dabei wurden Spaltenbreiten, Ausrichtungen und Beschriftungen angepasst, um die Lesbarkeit der Tabellen zu verbessern.
16.06.2026	15:00–17:00	Es wurde eine Funktion zur Berechnung von Tabellenprognosen entwickelt. Die Anwendung analysiert die aktuellen Tabellenstände und berechnet mögliche Endplatzierungen der Mannschaften.
17.06.2026	09:00–12:00	Die berechneten Prognosedaten werden in der Datenbank gespeichert. Dadurch können verschiedene Prognosestände dokumentiert und zu einem späteren Zeitpunkt miteinander verglichen werden.
17.06.2026	13:00–17:00	Eine Vergleichsfunktion wurde implementiert, die die Tabellenpositionen zweier Spieltage gegenüberstellt. Positionsänderungen werden farblich hervorgehoben und zusätzlich durch Symbole dargestellt.
18.06.2026	09:00–12:00	Da sich die Bundesliga während der Projektphase in der Sommerpause befand, wurden Testdaten erstellt. Diese ermöglichen die Simulation verschiedener Spielsituationen und Tabellenentwicklungen.
18.06.2026	13:00–17:00	Die Anwendung wurde um eine CSV-Exportfunktion erweitert. Dadurch können die angezeigten Tabellen in Microsoft Excel oder anderen Tabellenkalkulationsprogrammen weiterverarbeitet werden.
19.06.2026	09:00–12:00	Es wurden umfangreiche Funktionstests durchgeführt. Dabei wurden die Datenübernahme, die Prognoseberechnung, die Vergleichsfunktion sowie der CSV-Export überprüft.
19.06.2026	13:00–15:00	Während der Tests identifizierte Fehler wurden analysiert und behoben. Unter anderem wurde ein Darstellungsproblem in der Vergleichsfunktion untersucht und durch das Zurücksetzen der DataGridView-Spalten korrigiert.
19.06.2026	15:00–17:00	Abschließend wurden Screenshots erstellt und die Projektdokumentation fertiggestellt. Die Dokumentation beschreibt die Planung, Umsetzung, Testphase und die eingesetzten Technologien des Projekts.

6. Problembeschreibungen, Schwierigkeiten und Überarbeitungen

Während der Entwicklung traten verschiedene technische Probleme auf.

Ein Problem bestand bei der Verarbeitung von JSON-Daten mit JsonDocument. Da die Datenstruktur der API verschachtelt aufgebaut ist, mussten die einzelnen Eigenschaften mehrfach über GetProperty() ausgelesen werden.

Eine weitere Schwierigkeit trat bei der Arbeit mit DataGridView auf. Beim ersten Versuch wurden bestimmte Werte fälschlicherweise als Bilder interpretiert, was zu einer Laufzeitfehlermeldung führte. Nach der Überprüfung der Datenquellen und Spaltentypen konnte das Problem behoben werden.

Zusätzlich mussten verschiedene SQL-Abfragen mehrfach angepasst werden, um die gewünschten Tabellenstände und Prognosen korrekt zu berechnen.

8. Screenshots von Code-Snippets

```
using System.Text.Json;
using System.Threading.Tasks;

namespace BundesligaAnalyser
{
    2 Verweise
    internal class APIManager
    {
        // Dieser Code wurde mithilfe von ChatGPT entwickelt und anschließend angepasst
        1 Verweis
        public string GetJson(string league, string season)
        {
            // Daten aus der OpenLigaDB laden
            string url =
                $"https://api.openligadb.de/getmatchdata/{league.ToLower()}/{season}";

            HttpClient client = new HttpClient();

            try
            {
                string json = client.GetStringAsync(url).Result;
                return json;
            }
            catch
            {
                MessageBox.Show(
                    "Die OpenLigaDB konnte nicht erreicht werden.",
                    "Fehler"
                );
                return "";
            }
        }

        // Dieser Code wurde mithilfe von ChatGPT entwickelt und anschließend angepasst
        1 Verweis
        public void ReadJson(string json, string league, string season)
        {
            if (json=="")
            {
                return;
            }

            JsonDocument document = JsonDocument.Parse(json);
            DataBaseManager db = new DataBaseManager();
        }
    }
}
```

Der dargestellte Code zeigt die Anbindung der OpenLigaDB-API. Mithilfe der Klasse `HttpClient` werden aktuelle Bundesliga-Daten für die ausgewählte Liga und Saison im JSON-Format abgerufen. Anschließend werden die empfangenen Daten mit `JsonDocument` verarbeitet und die benötigten Informationen, wie Mannschaften, Spieltage und Ergebnisse, ausgelesen. Die Daten werden danach für die weitere Verarbeitung vorbereitet und in der SQLite-Datenbank gespeichert. Durch diese Lösung kann die Anwendung aktuelle Spieldaten automatisch aus einer externen Datenquelle übernehmen.

```

using Microsoft.Data.Sqlite;
using static System.Net.Mime.MediaTypeNames;

namespace BundesligaAnalyser
{
    12 Verweise
    internal class DataBaseManager
    {
        private const string connectionString =
            "Data Source=bundesliga.db";

        1 Verweis
        public void CreateDatabase()
        {
            SQLiteConnection connection =
                new SQLiteConnection(connectionString);

            string createTeams = @"
CREATE TABLE IF NOT EXISTS teams (
    id INTEGER PRIMARY KEY,
    season INTEGER,
    liga_id INTEGER,
    name TEXT NOT NULL
);";

            string createMatches = @"
CREATE TABLE IF NOT EXISTS matches (
    id INTEGER PRIMARY KEY,
    season INTEGER,
    liga_id INTEGER,
    home_team_id INTEGER,
    away_team_id INTEGER,
    home_goals INTEGER,
    away_goals INTEGER,
    matchday INTEGER,
    played INTEGER,
    match_datetime TEXT,
    FOREIGN KEY(home_team_id) REFERENCES teams(id),
    FOREIGN KEY(away_team_id) REFERENCES teams(id)
);";

            string createForecasts = @"
CREATE TABLE IF NOT EXISTS forecasts (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    season INTEGER,
    liga_id INTEGER,
    matchday INTEGER,
    team_id INTEGER,
    team_name TEXT,
    platz INTEGER,
    points INTEGER,
    position_range TEXT,
    created_at TEXT
);";

            try
            {
                connection.Open();

                SQLiteCommand cmd = connection.CreateCommand();

                cmd.CommandText = createTeams;
                cmd.ExecuteNonQuery();

                cmd.CommandText = createMatches;
                cmd.ExecuteNonQuery();

                cmd.CommandText = createForecasts;
                cmd.ExecuteNonQuery();
            }
            catch (Exception ex)
            {
                MessageBox.Show("Fehler beim Erstellen von Tabellen: "
                    + ex.Message);
            }
            finally
            {
                connection.Close();
            }
        }
    }
}

```

Der dargestellte Code zeigt die Erstellung und Initialisierung der SQLite-Datenbank. Über die Klasse `SQLiteConnection` wird eine Verbindung zur lokalen Datenbank hergestellt. Anschließend werden mithilfe von SQL-Befehlen die benötigten Tabellen für Mannschaften, Spiele und Prognosen angelegt, sofern diese noch nicht existieren. Die Tabellenstruktur definiert die erforderlichen Datenfelder sowie die Beziehungen zwischen den Datensätzen. Durch diese Vorgehensweise kann die Anwendung alle geladenen und berechneten Daten dauerhaft speichern und für spätere Auswertungen verwenden.

```

public DataTable PrognoseBerechnen(string league, string season_, int mday)
{
    DataTable table = new DataTable(); //
    SQLiteConnection connection =
        new SQLiteConnection(connectionString);
    int liga_Id = league == "BL1" ? 1 : 2;
    int season = int.Parse(season_);

    try
    {
        connection.Open();

        string abfrage = "...";

        SQLiteCommand cmd =
            new SQLiteCommand(abfrage, connection);

        cmd.Parameters.AddWithValue("@liga", liga_Id);
        cmd.Parameters.AddWithValue("@season", season);
        cmd.Parameters.AddWithValue("@matchday", mday);

        SQLiteDataReader reader = cmd.ExecuteReader();

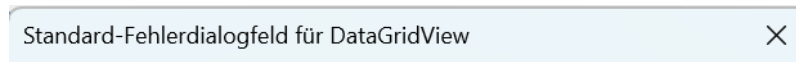
        table.Load(reader);
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message);
    }
    finally { connection.Close(); }

    return table;
}

```


Der dargestellte Code zeigt die Berechnung der Tabellenprognose. Zunächst wird eine Verbindung zur SQLite-Datenbank hergestellt und die ausgewählte Liga, Saison sowie der Spieltag als Parameter übernommen. Anschließend wird eine SQL-Abfrage ausgeführt, welche die für die Prognose benötigten Daten aus der Datenbank ermittelt. Die Ergebnisse werden mit einem SqlDataReader eingelesen und in ein DataTable-Objekt geladen. Das zurückgegebene DataTable dient anschließend als Grundlage für die Anzeige und Speicherung der berechneten Prognosedaten.

7. Screenshots vom Debugging

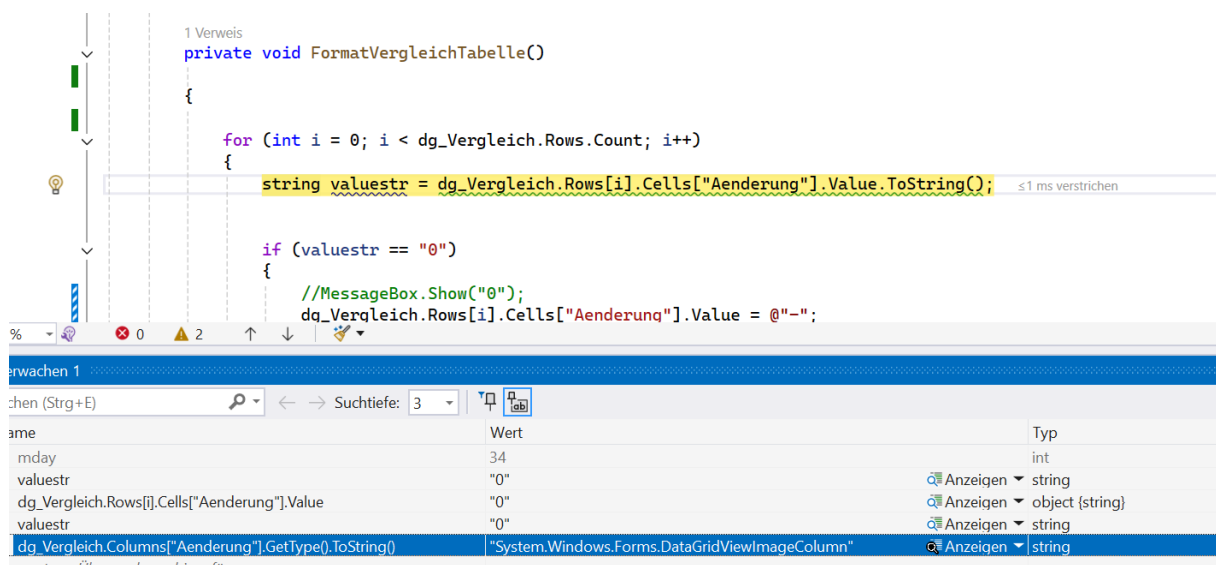


DataGridView-Ausnahme:

```
System.FormatException: Invalid cast from 'System.String' to
'System.Drawing.Image'.
---> System.InvalidCastException: Invalid cast from
'System.String' to 'System.Drawing.Image'.
   at System.Convert.DefaultToType(IConvertible value, Type
targetType, IFormatProvider provider)
   at System.Windows.Forms.Formatter.ChangeType(Object
value, Type type, IFormatProvider formatInfo)
--- End of inner exception stack trace ---
   at System.Windows.Forms.Formatter.ChangeType(Object
value, Type type, IFormatProvider formatInfo)
   at System.Windows.Forms.Formatter.FormatObject(Object
value, Type targetType, TypeConverter sourceConverter,
TypeConverter targetConverter, String formatString,
IFormatProvider formatInfo, Object formattedNullValue, Object
dataSourceNullValue)
   at
System.Windows.Forms.DataGridViewCell.GetFormattedValue(O
bject value, Int32 rowIndex, DataGridViewCellStyle& cellStyle,
TypeConverter valueTypeConverter, TypeConverter
formattedValueTypeConverter, DataGridViewDataErrorContexts
context)
```

Behandeln Sie das DataError-Ereignis, um dieses Standarddialogfeld zu ersetzen.

OK



Das Problem wurde gelöst, indem vor dem erneuten Zuweisen der Datenquelle die vorhandenen Spalten entfernt und die Datenquelle zurückgesetzt wurden:

```
dg_Vergleich.DataSource = null;  
dg_Vergleich.Columns.Clear();  
dg_Vergleich.DataSource = table;
```

8. Lösungsansätze

Für die Umsetzung des Projekts wurde eine Kombination aus C#, Windows Forms und SQLite gewählt. Diese Technologien ermöglichen eine einfache lokale Datenhaltung sowie eine übersichtliche grafische Benutzeroberfläche.

Die Trennung der Anwendung in die Klassen APIManager, DataBaseManager und Form1 verbessert die Wartbarkeit des Projekts. Darüber hinaus ermöglicht die Speicherung von Prognosen eine spätere Analyse und den Vergleich verschiedener Tabellenstände.

9. Quellenangaben

1. OpenLigaDB <https://www.openligadb.de>
2. OpenLigaDB API-Dokumentation <https://www.openligadb.de/api>
3. Microsoft Learn – .NET und C# Dokumentation <https://learn.microsoft.com/de-de/dotnet/>
4. Microsoft Learn – Windows Forms Dokumentation
<https://learn.microsoft.com/de-de/dotnet/desktop/winforms/>
5. Microsoft Learn – Microsoft.Data.Sqlite
<https://learn.microsoft.com/de-de/dotnet/standard/data/sqlite/>
6. SQLite Documentation <https://www.sqlite.org/docs.html>
7. ChatGPT (OpenAI) <https://chatgpt.com>

Verwendet zur Unterstützung bei der Entwicklung einzelner Programmabschnitte, Fehleranalyse und Dokumentation.

8. Eigene Projektnotizen, Testdaten und Entwicklungstests